

Web Basics - JavaScript

Lab Book

Copyright © Capgemini. All rights reserved.

No part of this publication shall be reproduced in any way, including but not limited to photocopy, photographic, magnetic, or other record, without the prior written permission of Capgemini Group. Capgemini Group considers information included in this document to be confidential and proprietary.

Document Revision History

Date	Revision No.	Author	Summary of Changes
12-May-2009	1.0	Pradnya Jagtap	Content Creation
05-Oct-2009	1.1	CLS Team	Review
24-Jun-2010	2.0	Anu Mitra	Refinements
03-May-2011	3.0	Karthik M/Anu Mitra	Integration Refinements
21-Apr-2015	4.0	Rathnajothi P	Revamp/Refinement
15-May-2015	5.0	Kavita D Arora	Integration Refinements
July – 2020	5.1	Neelima	Revamp/Refinement

Table of Contents

Document Revision History	2
Table of Contents	3
Getting Started	4
Overview	4
Setup Checklist for JavaScript	4
Instructions	4
Learning More (Bibliography if applicable)	4
Lab 1. Basics Concepts of JavaScript	5
1.1: Using Variable in many Script tags	5
Lab 2. The JavaScript Language	7
2.1: For loop in JavaScript	7
2.2: Create a web page to calculate the Compound Interest using the formula given below:	7
Lab 3. Working with Predefined core objects	9
3.1: Displaying Date using Date Object	9
3.2: Using indexOf function of String object	9
3.3: Using various String methods	10
Lab 4. Working with Arrays	11
4.1: Using Array to display values	11
Lab 5. Working with Document Object Model(DOM)	12
5.1: Window object	12
Lab 6. Working with Location Object	14
6.1: Location Object	14
Lab 7. Working with Document Object	15
7.1: Working with Documents	15
Lab 8. Working with Form Object	19
8.1: Form Validation	19
8.2 Validate Field	23
Appendices	25
Appendix A: JavaScript Standards	25
Appendix B: Coding Best Practices	30

Getting Started

Overview

These Lab book is a guided tour for Learning JavaScript. It contains solved examples and To Do assignments. Follow the steps provided in the solved examples and then work out the 'to do' Assignments given.

Setup Checklist for JavaScript

Here is what is expected on your machine in order for the lab to work.

Minimum System Requirements

- Hardware: Networked PCs with minimum 64 MB RAM and 60 MB HDD.
- Software: Window based Operating System having the latest version of Internet Explorer (IE) or Netscape Navigator installed.

Please ensure that the following is done:

- A text editor like Notepad, Eclipse Luna or Visual Studio 2008 is installed.

Instructions

- For coding standards refer Appendix – A.
- All Lab assignments should follow the coding standards.
- Create a directory by your name in drive <drive> for JavaScript assignments.
- In this directory, create subdirectory javascript_assgn.
- For each lab create directory as lab<lab number>.

Learning More (Bibliography if applicable)

- Beginning JavaScript by Paul Wilton
- JavaScript: The Definitive Guide by David Flanagan
- JavaScript Application Cookbook by Jerry Bradenbaugh

Lab 1. Basics Concepts of JavaScript

Goals	Learn to embed script tags in different parts of the HTML document.
Time	15 minutes

1.1: Using Variable in many Script tags

Solution:

Step 1: Create **Prob5.html** page, and complete the following code and save it in lab1 directory.

```
<html>
<head><title>Embedding Script tag in HTML Document</title>

<script>
/*
TODO:define variable headVar and initialize it to some integer value and display the value as
shown in the Fig 6
*/
</script>

<hr>
</head>
<body>

<script>
/*
TODO:define variable bodyVar and initialize it to some integer value and display the value as
shown in the Fig 6
*/
</script>

<hr>
<script src="common.js">
</script>

<script>
/*
TODO: Invoke the method addNos(headVar,bodyVar) defined in common.js file and pass
the two variables headVar and bodyVar defined in the head and the body script tag and
display the added result as shown in the Fig 6
*/
</script>

<hr>
</body>
</html>
```

Example: Lab 1: Prob5.html

Step 2: Create a file **common.js** which has a function **addNos()** that adds two numbers and returns the addition of two numbers.

```
var msg;  
msg="<p><code>The actual script is in external script file called common.js</code></p>";  
  
function addNos(headVar,bodyVar)  
{  
  
    //TODO: display the contents of the variable "msg"  
  
    //TODO: display the addition of two numbers  
  
}
```

Example : Lab 1: common.js

Step 3: Open **prob5.html** page in the browser, and verify that you get the same output as required.

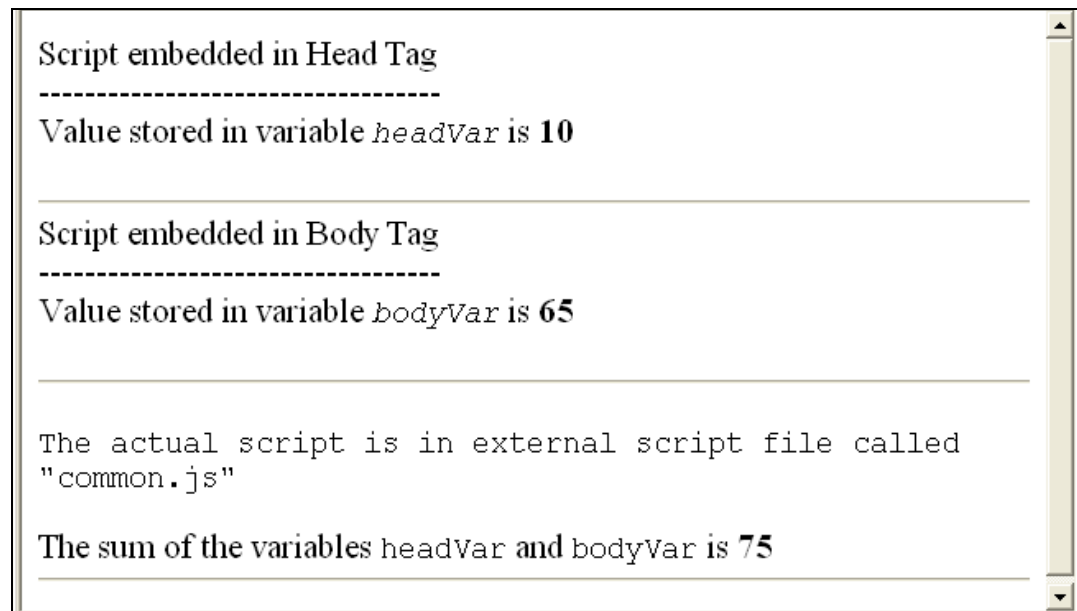


Figure : Using Variable in many Script tags

Lab 2. The JavaScript Language

Goals	Learn to use looping structures and operators in JavaScript.
Time	20 minutes

2.1: For loop in JavaScript

Create a web page containing a heading “Layout is here” followed by a horizontal rule and a table with a single row as shown in the figure given below.

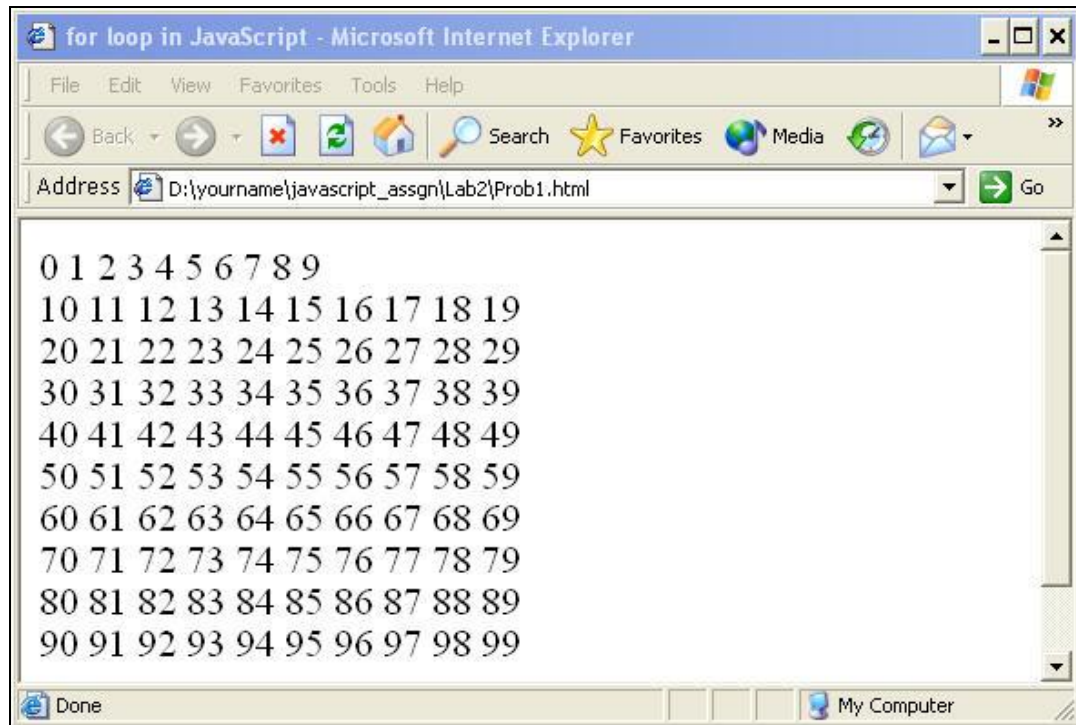


Figure : For loop in JavaScript

After completing the loop, the variable used, that is “I”, should be equal to 100.

2.2: Create a web page to calculate the Compound Interest using the formula given below:

$$\text{Compound Interest} = \left[P * \left(1 + \frac{r}{100} \right)^n \right] - P$$

Where:

p = Principal,

r = Rate of Interest,

n = period in years

The values used in the example in the following figure are as follows:

P = 1000, n = 1, r = 10

*****Calculate Compound Interest*****		
Prinicipal	-	1000 rs
Rate of Interest	-	10%
Period	-	1 yr
Comp Interest	-	100

Figure : Operators and Arithmetic Expression

Lab 3. Working with Predefined core objects

Goals	- Understand Date, String Object - Learn to use Date and String objects in HTML pages
Time	20 minutes

3.1: Displaying Date using Date Object

Create a web page **Prob1.html** with a heading and a paragraph describing the impact of COVID-19.

- There should be only one heading, use a H1 tag for the heading.
- There should be only one description section, the description should be inside a P tag after the heading tag
- Date format** should be YYYY/MM/DD
- Also greet the user depending on the time the user visits the page.

Time	Msg to be displayed
< 12	Good Morning
>= 12 and <= 17	Good Afternoon
> 17	Good Evening

3.2: Using indexOf function of String object

Create a web page **prob2.html**, which uses the **indexOf** method of string object and displays the index number of the substring searched for within the string.

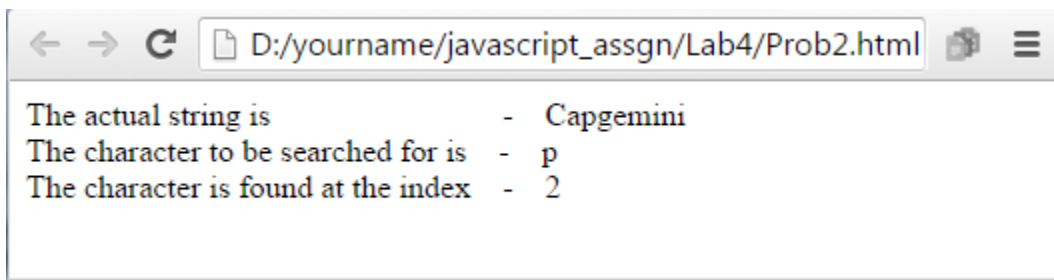


Figure : Using indexOf method of String object

3.3: Using various String methods

Write **prob3.html** page by completing the following code that demonstrates some of the methods of the String objects like **match**, **substr**, **lowerCase**, and **upperCase** to produce the output as shown in the figure given below:

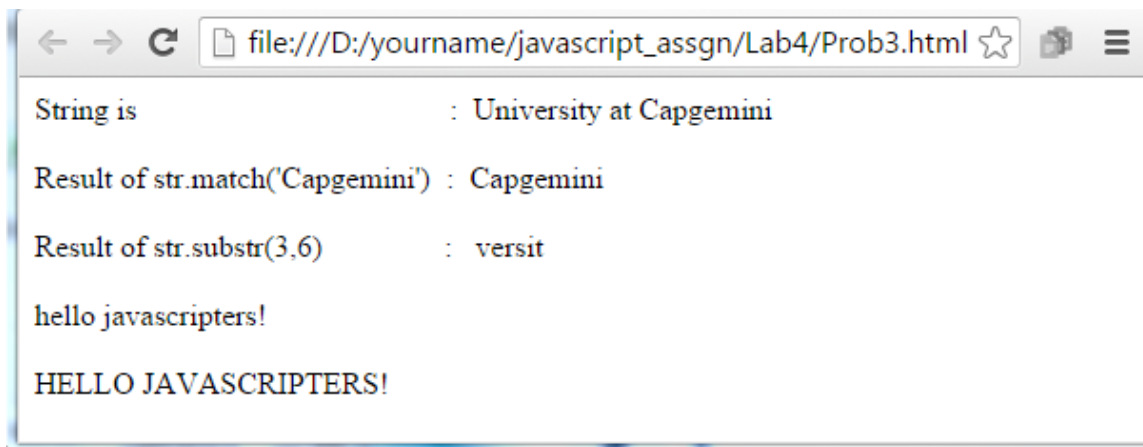


Figure : Using various String methods

Lab 4. Working with Arrays

Goals	• Work with Array Object
Time	5 minutes

4 .1: Using Array to display values

Create a **prob1.html** web page containing script. In this script, declare an array of 6 employee names and display it in the browser as shown below:



Figure : Using Array to display values

Lab 5. Working with Document Object Model(DOM)

Goals	• Understand Window Object. • Dynamically create windows • Handle window events.
Time	20 minutes

5.1: Window object

Create a **prob1.html** web page which has the following items as shown in the figure given below:

- a form that accepts window parameters width, height, title, left and top parameters from text field, and
- two buttons with the labels **New Window** and **Reset** to the web page

The screenshot shows a web browser window with the address bar displaying 'D:\yourname\javascript_assgn\Lab6\Prob1.html'. The main content area contains a form with the following structure:

- A title box containing the text "Create a new window".
- A section titled "Window Size" containing two text input fields: "Width:" and "Height:".
- A section titled "POSITION" containing two text input fields: "Left:" and "Top:".
- At the bottom of the form are two buttons: "New Window" and "Reset".

The browser's status bar at the bottom shows "Done" and "My Computer".

Figure : Interface to accept window coordinates

If **Reset** button is clicked, then clear all text fields. If **New Window** button is clicked, then open a new window with specifications entered in the text fields as shown in the figure given below.

Note: By default, the new window opens at the top left corner of the screen.

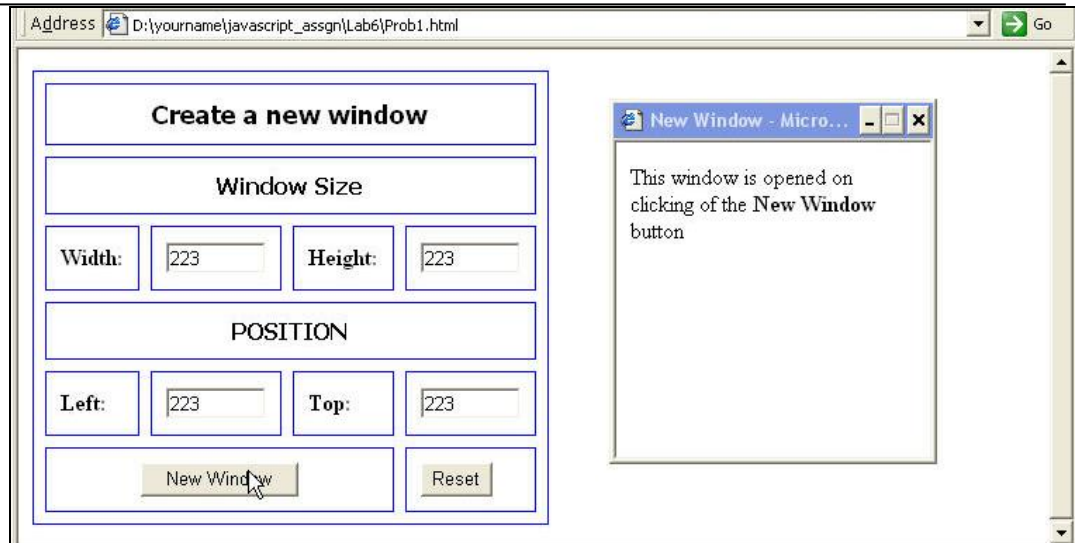


Figure : Opening a window

Solution:

Complete the following Code and save it as **Prob1.html**.

```
<html>
<head>
<title> window example </title>
</head>
<script>
function nwindow()
{
/*TODO: get the height, width, left and top from the form object and pass the values to
open method of window along with the name of the html file to be opened in the new
window.*/
}
</script>
<body >
<form id="frmlab">
<table border="1" cellspacing="8" cellpadding="10" bordercolor="blue">

// Create Table as shown in fig 6.2
</table>
</form>
</body>
</html>
```

Example : Lab 5: Prob1.html

Lab 6. Working with Location Object

Goals	Understand and use Location Object.
Time	10 minutes

6.1: Location Object

Create a web page which will display the properties **href**, **protocol**, and the **pathname** of the location object of your current file.

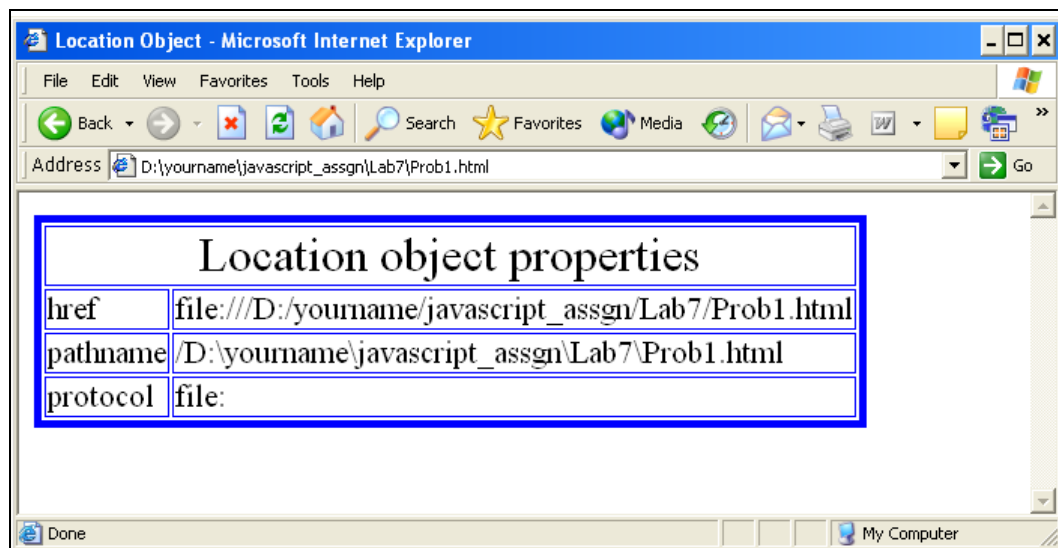


Figure 1: Location Object Properties

Lab 7. Working with Document Object

Goals	Understand Document Object
Time	30 minutes

7.1: Working with Documents

Create a **prob1.html** web page which displays products available as shown in the following figure. The product details comprise Product Name, Product description, and its price.

Name	Description	Price	Qty
Barbie Doll	Beautiful	20	
Calculator	Calculator with latest features	30	
Mobile Phone	Camera, Java Games, GPRS	40	
LG DVD	5 disc changer	50	

Figure 2: Displaying Products

Users can place orders specifying the quantity of each product. If the user does not enter quantity in any of the text fields, then an error message should be displayed as shown in the figure given below:



Figure 3: Validating Products

When the user clicks the **Order** button, the invoice for the current products transaction showing the product name, quantity ordered, price and total amount is displayed in a new window as shown in the figure given below:

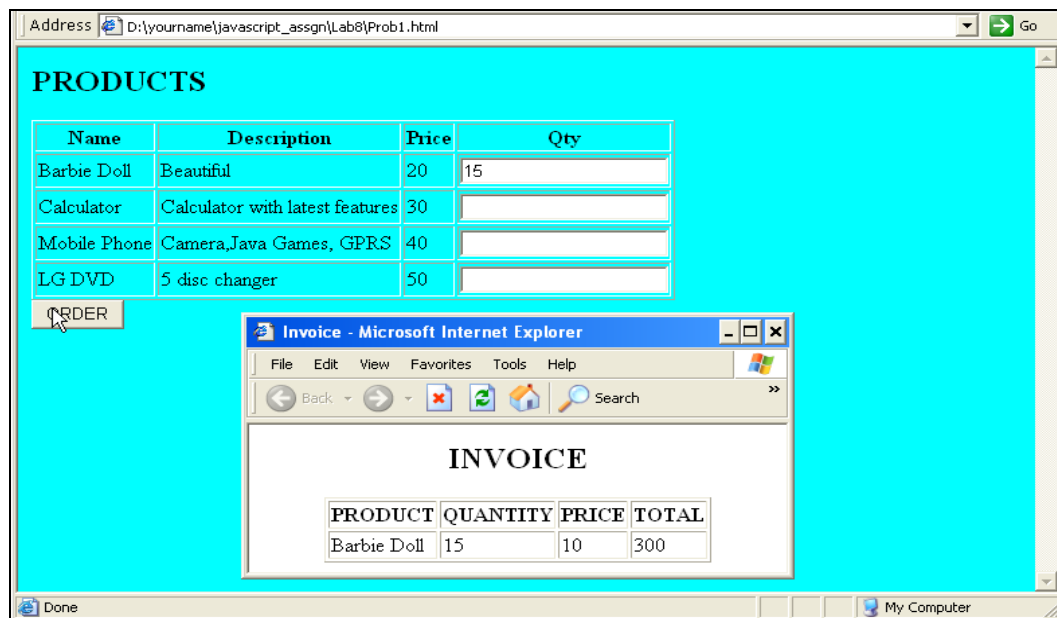


Figure 4: Displaying Invoice details in a new window

Solution:

Step 1: Write the code and save it as **Prob1.html**.

Step 2: Open **prob1.html** page in the browser, and verify that you get the same output as required.

7.2: Create a Calculator

HTML Specifications :

- There should be a main div with class “calculator-frame”.
 - Enclosed in this div there should be a div with class “output-tray”
 - Enclosed in this div there should be a div with class “output-display-holder”
 - Enclosed in this p tag there should be a div with class “output-display”
 - After this, there should be an input tag of type “hidden” and value “0”. The id is “expression-string”
 - After this div, there should be a div with class “button-tray”
 - Enclosed in this div there should be a div with class “button-group”.
 - Enclosed in this div there should be a div with class “calculator-button” with innerHTML “1”
 - After this, div there should be a div with class “calculator-button” with innerHTML “2”
 - After this, div there should be a div with class “calculator-button” with innerHTML “3”
 - After this, there should be a div with class “button-group”.
 - Enclosed in this div there should be a div with class “calculator-button” with innerHTML “4”
 - After this, div there should be a div with class “calculator-button” with innerHTML “5”
 - After this, div there should be a div with class “calculator-button” with innerHTML “6”
 - After this, there should be a div with class “button-group”.
 - Enclosed in this div there should be a div with class “calculator-button” with innerHTML “7”
 - After this, div there should be a div with class “calculator-button” with innerHTML “8”
 - After this, div there should be a div with class “calculator-button” with innerHTML “9”
 - After this, there should be a div with class “button-group”.
 - Enclosed in this div there should be a div with class “calculator-button” with innerHTML “0”
 - After this, div there should be a div with class “calculator-button” with innerHTML “/”
 - After this, div there should be a div with class “calculator-button” with innerHTML “+”
 - After this, there should be a div with class “button-group”.
 - Enclosed in this div there should be a div with class “calculator-button” with innerHTML “-”
 - After this, div there should be a div with class “calculator-button” with innerHTML “*”
 - After this, div there should be a div with class “calculator-button” with innerHTML “=”

CSS Specifications

- Class “calculator-frame” should have the following rules:
 - Height should be 320px;
 - Width should be 280px;
 - Border should be 1px solid and silver in color
 - It should have absolute position
 - It should be positioned 6px from top and 40% from the left
 - border radius should be 7px
- Class “output-tray” should have the following rules:
 - Position should be relative
 - Height should be 100px
 - The background color should be slategray
- Class “output-display-holder” should have the following rules:
 - Position should be absolute
 - It should be positioned 0px from bottom
 - The width should be 100%
 - The height should be 70px
 - The color should be of hex value `#33FF00`
 - The font size should be 30px
 - The text should be aligned to the right
- Class “button-tray” should have the following rules:
 - The margin from top should be 10px
- Class “calculator-button” should have the following rules:
 - The margin from the left should be 5px
 - The width should be 30%
 - The height should be 20px;
 - Float should be left
 - The text should be aligned to the center
 - The border should be 1px solid silver
- Class “button-group” should have the following rules:
 - The margin from top should be 15px
 - The height should be 22px

Javascript Description

There should be a listener when on clicking on any of the calculator buttons will behave in the following way:

If the button pressed is “=”, then you should evaluate the operator string buffered till now in the input tag’s value whose id is “expression-string”

Otherwise, you should update the expression String accordingly. Also on each click, the output of the calculator should be changed accordingly.



Lab 8. Working with Form Object

Goals	Understand and use Form Object.
Time	45 minutes

8.1: Form Validation

Design a Conference room booking form with the following specifications.

- There should be a main div with class "main-form". Hereafter, we will refer to it as main-form
 - Enclosed in the main-form is a div with class "form-element" with id "form-heading".
 - Enclosed in this div is another div whose innerText is "Conference Booking Form".
 - Below the "form-heading" div, we have 6 divs with class "form-element". They are described in order below:
 - in the first div with class "form-element",
 - there is a div enclosed with class "form-element-child" and "input-label".
 - Enclosed inside this div is a p tag with text "First Name".
 - After the div described just above, we have another div with class "form-element-child" and "input-validation".
 - Enclosed inside this div is an input tag with class "user-input" with type="text" and id="first-name"
 - After the input tag described just above, we have another div with class "error-container" and "disabled" with innerText "Error". It has the id "first-name-error"
 - in the second div with class "form-element",
 - there is a div enclosed with class "form-element-child" and "input-label".
 - Enclosed inside this div is a p tag with text "Last Name".
 - After the div described just above, we have another div with class "form-element-child" and "input-validation".
 - Enclosed inside this div is an input tag with class "user-input" with type="text" and id="last-name"
 - After the input tag described just above, we have another div with class "error-container" and "disabled" with innerText "Error". It has the id "last-name-error"
 - In the third div with class "form-element",
 - there is a div enclosed with class "form-element-child" and "input-label".
 - Enclosed inside this div is a p tag with text "Email".
 - After the div described just above, we have another div with class "form-element-child" and "input-validation".
 - Enclosed inside this div is an input tag with class "user-input" with type="text" and id="email"

- After the input tag described just above, we have another div with class "error-container" and "disabled" with innerText "Error".It has the id "email-error"
- In the fourth div with class "form-element",
 - there is a div enclosed with class "form-element-child" and "input-label".
 - Enclosed inside this div is a p tag with text "Mobile Number".
 - After the div described just above, we have another div with class "form-element-child" and "input-validation".
 - Enclosed inside this div is an input tag with class "user-input" with type="text" and id="mobile-number"
 - After the input tag described just above, we have another div with class "error-container" and "disabled" with innerText "Error".It has the id "mobile-number-error"
- in the fifth div with class "form-element" and id="date-element" the structure is as follows:
 - there is a div with class "form-element-child" and "start-date"
 - enclosed inside this div is another div with innerText "Start Date"
 - below the described div above, we have div.
 - Enclosed inside this div, we have an input tag with id as "start-date" and type="date"
 - after the div described above(See the nearest point with same indent), we have a div with class "error-container" and id="start-date-error" with innerText "Error".
 - after the div described above, there is a div with class "form-element-child" and "end-date"
 - enclosed inside this div is another div with innerText "End Date"
 - below the described div above, we have div.
 - Enclosed inside this div, we have an input tag with id as "end-date" and type="date"
 - after the div described above(See the nearest point with same indent), we have a div with class "error-container" and id="end-date-error" with innerText "Error".
- In the sixth div with class "form-element", we have an anchor tag with href "#" and id "submit-button" with innerText "Submit"

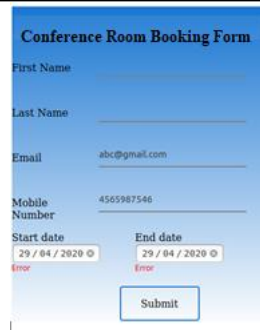
CSS Required

- the "input-label" class has the following rules:
 - width is 30%
 - the "form-element-child" class has the following rules:
 - float is left
 - height is inherit
- the "input-validation" class has the following rules:
 - width is 65%
 - it has left margin as 15px
- the "form-element" class has the following rules:
 - height is 70px

- the "error-container" class has the following rules:
 - it has red color
 - it has 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif as font-family
 - it has 12px as font-size
- all the input tags with type text in elements with "main-form" class have the following rules:
 - it has border none
 - the height is 40px
 - the background image is none
 - the background color is transparent
 - the webkit-box-shadow is none
 - the moz-box-shadow is none
 - the box-shadow is none
 - the border-bottom is 2px solid grey
 - it has a transition of 0.8s with transition-timing-function as linear
- the "main-form" class has the following rules:
 - the background-image is a linear gradient with starting color "#1976d2" and stopping color white.
 - the width is 400px
 - it has margin-left and margin-right is 50%
 - it has border as 1px solid with color "#1976d2"
- the "disabled" class has the following rules:
 - display is none
- the rules for behaviour of all input tags inside container with class "main-form" with type as "text" on focus are given below:
 - the bottom border is 2px solid with color "#115293"
 - the "date-element" id has the following rules defined:
 - height is 80px
- all the anchor tags inside container of class "form-element" are defined by rules given below:
 - color is black
 - margin left is 42% and margin-right is 50%
 - text decoration is none
 - height is 50px
 - line-height is 50px
 - width is 30%
 - text-align is center
 - border is 2px solid with color "#1976d2"
 - border radius is 4px
 - top margin is 5px
 - display is inline-block
 - it has a transition of 0.8s with transition-timing-function as ease
- all anchor tags in containers with form-element class on hover behave as follows:
 - background color is #1976d2
- the "start-date" and "end-date" class has the following rules:
 - width is 48%

JavaScript requirements:

- You have to add and EventListener to the **Submit** button, defining the following behaviour:
 - You have to perform validation for email, mobile number, and some basic validations for start date and end date
 - The email should follow a pattern. **Hint :** you may use the pattern “(?:[a-z0-9!#\$%&'*/=?^_`{|}~]+(?:\.[a-z0-9!#\$%&'\*/=?^\_`{|}~]+)*)\"(?:[x01-x08\x0b\x0c\x0e-x1f\x21\x23-\x5b\x5d-x7f]|\\[\x01-\x09\x0b\x0c\x0e-x7f])*)\"@\"(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\\[(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.]\"{3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9])?[a-z0-9-]*[a-z0-9](?:[x01-x08\x0b\x0c\x0e-x1f\x21-\x5a\x53-x7f]|\\[\x01-\x09\x0b\x0c\x0e-x7f])+\.)\"” for validation of email
 - The mobile number should follow a pattern. **Hint:** You may use the pattern “[1-9]{1}[0-9]{9}” for validation of mobile number
 - Email Validation
 - If the email is empty or incorrect, you have to enable the email error container div(with class “email-error”) and populate the message “Invalid Email”.
 - If the email is correct, you have to disable the email error container.
 - Mobile Number Validation
 - If the mobile number is empty or incorrect, you have to enable the mobile number error container div(with class “mobile-number-error”) and populate the message “Invalid Mobile Number”
 - If the mobile number is correct, you have to disable the mobile number error container.
 - Start Date and End Date validation
 - Both the start date and end date should resemble valid dates. You may use the Date() JS API to do the same
 - If the start date is not valid, you have to enable the div with class “start-date-error” and populate the div with text “Start Date is not entered correctly”
 - If the start date is valid, you have to disable the div with class “start-date-error”
 - If the end date is not valid, you have to enable the div with class “end-date-error” and populate the div with text “ End Date is not entered correctly”
 - If the end date is valid, you have to disable the div with class “end-date-error”.
 - If both start date and end date are valid, you have to check if end date is greater than or equal to the start date. If it is, you have to disable both the ‘start-date-error’ and ‘end-date-error’ div classes
 - Otherwise, enable the div with class “start-date-error” and populate the message “End Date has to be greater than or equal to the Start Date.”
- If there is no error, you have to change the text of the submit button and change it to “Success!!!”.
- Otherwise, if there exists any error, you have to change the text of the submit button temporarily to “Error!!!”. After exactly 3 seconds, the text of the submit button should revert back to “Submit”. Javascript’s **setTimeout()** may come in handy



Conference Room Booking Form

First Name

Last Name

Email

Mobile Number

Start date End date

Fig: Webpage

8.2 Validate Field

Create a **prob2.html** page as shown in the below figure.



Product Details

Category:

Product:

Quantity:

Total Price:

Figure 20: Lab 8.2 Product Details

Data should be prepopulated in category list box (Electronics, Grocery). Based on selection of category, product list need to be populated automatically with values as given in the below table. Also Total price need to be calculated for the entered quantity as per the data in the below table. Total price field should be non-editable field.

Category	Product	Price per quantity in Rupees
Electronics	Television	20000
	Laptop	30000
	Phone	10000
Grocery	Soap	40
	Powder	90

While clicking on submit button, if all the text fields contains valid values then display the filled details in a popup window.

Appendices

Appendix A: JavaScript Standards

1: Naming conventions for variables in JavaScript:

- Variables must begin with prefix indicating the type of the variable.
 - All integer must start with "int".
 - All floating data types must start with "flt".
 - All string must start with "str".
 - All object name must start with "obj".
 - All Boolean variables must start with "bln".
 - All variables that store date must start with "dt".
 - All constants must be in upper case with different words separated by underscore (_).
 - All array variables must start with "arr".
 - Apart from these guidelines all variable name must be sensible enough, so that it's purpose can be identified from it's name.

Type	Example
String	strStringName
Boolean	blnPresent
Array	arrArrayName
Object	objObjectName
Date	dtDateName
Integer	intValueInteger
Float	fltValueFloat
Constants	STRING_CONSTANT ARRAY_CONSTANT NUMERIC_CONSTANT

- All HTML elements must be prefixed with appropriate types.
 - TextBox "txt"
 - Image "img"
 - Image map Area "img"
 - option button "opt"
 - CheckBox "chk"
 - DropDown List "lst"

- Form Name "frm"
- Buttons "btn"
- All div tags "div"
- All class names must start with "cls"
- All user-defined objects must start with "u"
- First letter of each variable/function name must be in upper case. Rest all letters must be in lowercase.
- Use of underscore and digits for naming variables must be avoided.

Tag	Example
Div	divContent
Class	clsInterest
Form	frmContainer
Image	imgMapThis
Button	btnOk
TextBox	txtInterestRate
CheckBox	chkAllow
Option Button	rdbRate
DropDownList	lstState

It must be noted that this naming style does not apply to HTML elements. However, when these elements are accessed in the JavaScript functions, these naming conventions must be followed. This document describes coding convention only for JavaScript.

2: Commenting

- Comments related to a particular line of code should be on the same line after the statement gets over.

```

If (dtToday == "15/07/99") {           // Is date birthdate?
    alert ("Happy Birthday");           // Give birthday message
} else {
    alert ("Happy Day");                 // Give standard message
}

```

- Over all commenting should consist of two parts – Comment header and Comment footer. Comment header must precede the block of code and Comment footer must follow the block of code.

```
//Function Name: calculateInterest
//Description: This function calculates the interest. It accepts the initial investment
//              and period for which the amount is invested. Rate of interest is
//              fixed. Formula is
//              
$$\text{fltInterest} = \text{fltAmount} * \text{fltPeriod} * \text{fltRATE}/100$$

//              Dhrumil Dalal
//              15/07/1999
//Author:       fltAmount – indicated the amount invested
//Start Date:   fltPeriod – Indicates the period of investment
//Input        Calculated interest
Parameters:
//
//Return Value:
Function calculateInterest(fltAmount,fltPeriod){
}

//End of function for calculating the rate of interest
```

3: Documentation

- All variables used in the function must be declared in brief.
- Only one variable declaration per line.
- Describe each variable on the same line and description should not be more than one line.
- All functions must be preceded by comments. Comments must describe the following:
 - Input parameters.
 - Return value.
 - Function logic in brief.
 - Starting date.
 - Name of the author.
 - Revision history.
- After the end of function, there must be block of comment indicating the end of function.

```
//Function Name: calculateInterest
```

```
//Description:      This function calculates the interest. It accepts the initial
//                  investment and period for which the amount is invested.
//                  Rate of interest is fixed. Formula is
//                  fltInterest = fltAmount * fltPeriod      *fltRATE/100
//                  Dhrumil Dalal
//                  15/07/1999
//                  fltAmount – indicated the amount invested
//Author:           fltPeriod – Indicates the period of investment
//Start Date:      Calculated interest
//Input Parameters:
//
//Return Value:
Function calculateInterest(fltAmount,fltPeriod){
Var fltRATE = 12.5; // fixed rate of interest
Var fltInterest; // The variable to store calculated interest
fltInterest = fltAmount * fltPeriod * fltRATE/100 ;
return fltInterest;
}

//End of function for calculating the rate of interest
```

4: Coding Styles

- For statements which may have block of code enclosed in {}, the opening brace "{" must immediately follow the statement and the closing brace "}" must be below the statement. That is to say, the closing brace and first letter of the statement must be same in the column.

```
If (condition) {
...
} else {
  if (condition) {
    ...
  } else {
    ...
  }
}

for(intCounter=0; intCounter <= 5; intcounter++){
  //Perform calculation.
  //Display Result}
```

- All statements within corresponding opening and closing brace must be indented. Indentations must be in odd columns.

Column no

```
123456789.....  
if (condition) {  
    ...  
} else {  
    if (condition) {  
        ...  
    } else {  
        ...  
    }  
}
```

- Also the code should not extend past the 80th column so that it is required to scroll to the right or left to edit a particular line. In the case of strings which do not fit on one line, it is recommended that temporary variables be used with the string concatenation operator (+) to construct strings of longer lengths. The following example illustrates this:

```
Column no  
123456789.....80  
strMessage = "Demonstrating the use of ... ";  
strMessage += "prepared on 15-07-1999";
```

Appendix B: Coding Best Practices

The following demonstrate the best practices that should be followed while writing JavaScript code.

1: Inline JavaScript source code

Any JavaScript code that does not write out to the document should be placed within the head of the document.

```
<HTML>
<HEAD>
<SCRIPT>
<!--
function functionName() {
    alert(text);
}

var text = 'Hello World';
//-->
</SCRIPT>
</HEAD>
```

Example 1: Sample Code

This ensures that the browser has loaded the JavaScript function definitions before it is required. It also makes it slightly easier to maintain the JavaScript code if it can always be found in the head of the document.

2: JavaScript Links

Avoid using the **javascript:** protocol as a default URL within a link.

If JavaScript is disabled, then the link will not work. Do not use the following:

```
<SCRIPT>
<!--
function functionName() {
    alert('Hello world');
}
//--></SCRIPT>
<A HREF="javascript:functionName()">text link</A>
```

Example 2: Sample Code

Instead, use JavaScript itself to override the **href** property of the link:

```

<SCRIPT>
<!--
function functionName() {
    alert('Hello world');
}
//-->
</SCRIPT>

<A HREF="default.htm" onClick="this.href='javascript:functionName()'">text link</A>

```

Example 3: Sample Code

3: Avoid Using Void

All browsers do not support the **void** function. Create your own **void** function.

The in built **void()** function is supported since JavaScript 1.1. Therefore it is best to create your own void function rather than rely on JavaScript 1.1 being available.

```

<SCRIPT>
<!--
function myVoid() { } // create a void function
//-->
</SCRIPT>

<A HREF="#" onClick="this.href='javascript:myVoid()'">non functional text link</A>

```

Example 4: Sample Code

4: JavaScript Performance

Avoid writing output multiple times to the document, concatenate the data, and then write all in one go.

With the introduction of Netscape Navigator 4, the rendering of JavaScript generated HTML slowed down considerably.

The following writes the HTML output to the document in one go:

```

<SCRIPT>
<!--
var output = '<P>';
output += 'Last modified: ';
output += document.lastModified;
output += '</P>'
document.write(output);
//-->
</SCRIPT>

```

Example 5: Sample Code

5: Select Form Fields

Use the Netscape method to correctly navigate select field properties.

The following technique works in Microsoft Internet Explorer. However it should be avoided.

```
<SCRIPT>
<!--
var property = document.formName.selectName.propertyName
//-->
</SCRIPT>
```

Example 6: Sample Code

Whereas the following will work correctly in all browsers:

```
<SCRIPT>
<!--
var property =
document.formName.selectName.options[document.formName.selectName.options.selectedIndex].propertyName
//-->
</SCRIPT>
```

Example 7: Sample Code

6: Changing Location

Do not use the following:

```
<SCRIPT>
<!--
location = 'page.htm';
//-->
</SCRIPT>
```

Example 8: Sample Code

The later approach is confusing as it is not clear whether you are changing the location property of the “window” or the “document object”.

Changing the location using the document is deprecated and causes problems on later browsers. Use the following:

```
<SCRIPT>
<!--
window.location.href = 'page.htm';
//-->
</SCRIPT>
```

Example 9: Sample Code

7: Opening Windows

While opening a new popup window using JavaScript, there are several points to bear in mind.

To be able to control the popup window from the **opener** window, always retain the returned reference from the window's open method:

```
<SCRIPT>
<!--
var windowHandle = window.open('page.htm','windowName','width=600,height=320');
//-->
</SCRIPT>
```

Example 10: Sample Code

To avoid errors while referring to the **opener** window from the **popup** window, always check for the in-built browser support for the **opener** property. If necessary, provide your own:

```
<SCRIPT>
<!--
var windowHandle = window.open('page.htm','windowName','width=600,height=320');
if (!windowHandle.opener)
    windowHandle.opener = self;
//-->
</SCRIPT>
```

Example 7: Sample Code

While updating the contents of a newly opened window, give the browser time to open the window and to load the initial contents:

```
<SCRIPT>
<!--
function update() {
    windowHandle.document.open();
    windowHandle.document.write('<H1>Hello World</H1>');
    windowHandle.document.close();
}

var windowHandle = window.open('page.htm','windowName','width=600,height=320');
if (!windowHandle.opener)
    windowHandle.opener = self;
setTimeout('update()',2000);
//-->
</SCRIPT>
```

Example 12: Sample Code

8: JavaScript Entities

JavaScript Entities are only supported by Netscape Navigator. Avoid their use.

The following will cause errors in other browsers:

```
<HR WIDTH="{barWidth}%">
```

Example 13: Sample Code

Instead the following can be used.

```
<SCRIPT>
<!--
document.write('<HR WIDTH="' + barWidth + '%">');
//-->
</script>
```

Example 14: Sample Code